AI ENGINEER GEMINI EVENT

RPG: Robotic Planning with Gemini

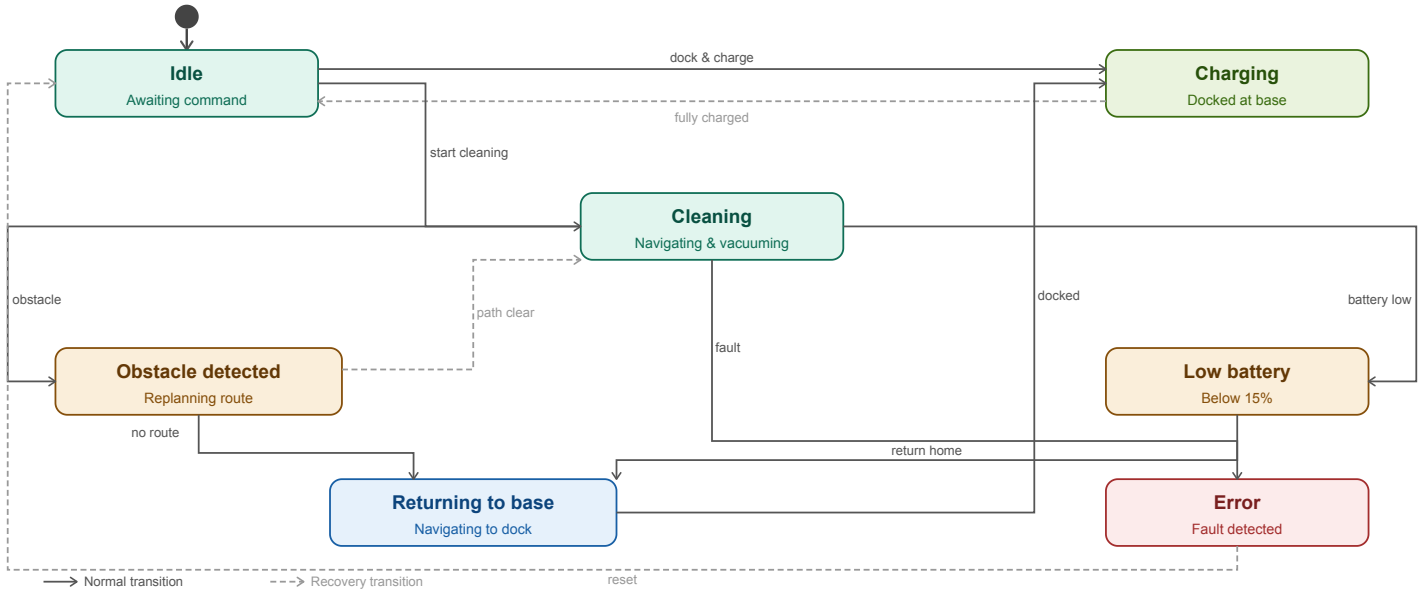
Anurag Roy · Chaitanya Jadhav

Why Tackle Mapping?

- Map-building is computationally expensive and environment-specific
- Maps may become stale quickly
- Each new deployment environment requires a fresh mapping pass

Source: Hess et al., Real-time loop closure in 2D LIDAR SLAM

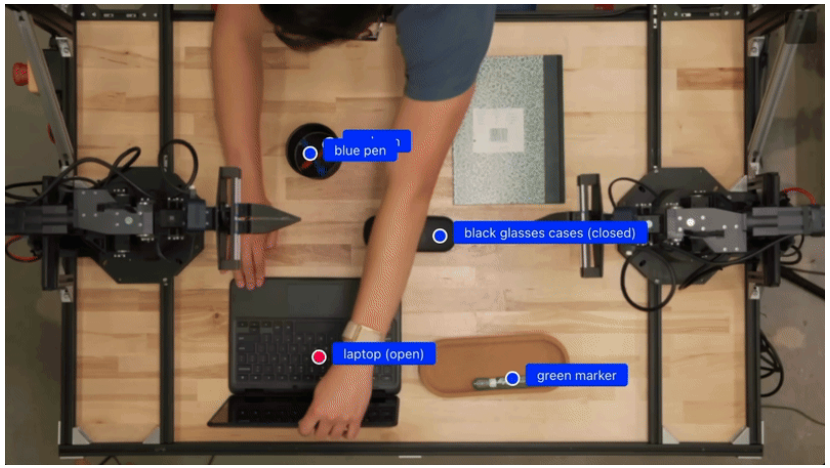
Why Tackle Planning?



Traditional approaches use state machines, which require explicit understanding of the environment and manual setting of waypoints.

How LLMs and VLAs Change the Game

- Models already understand rooms, objects, and spatial relationships
- Therefore, they can be used for both dynamic mapping and planning

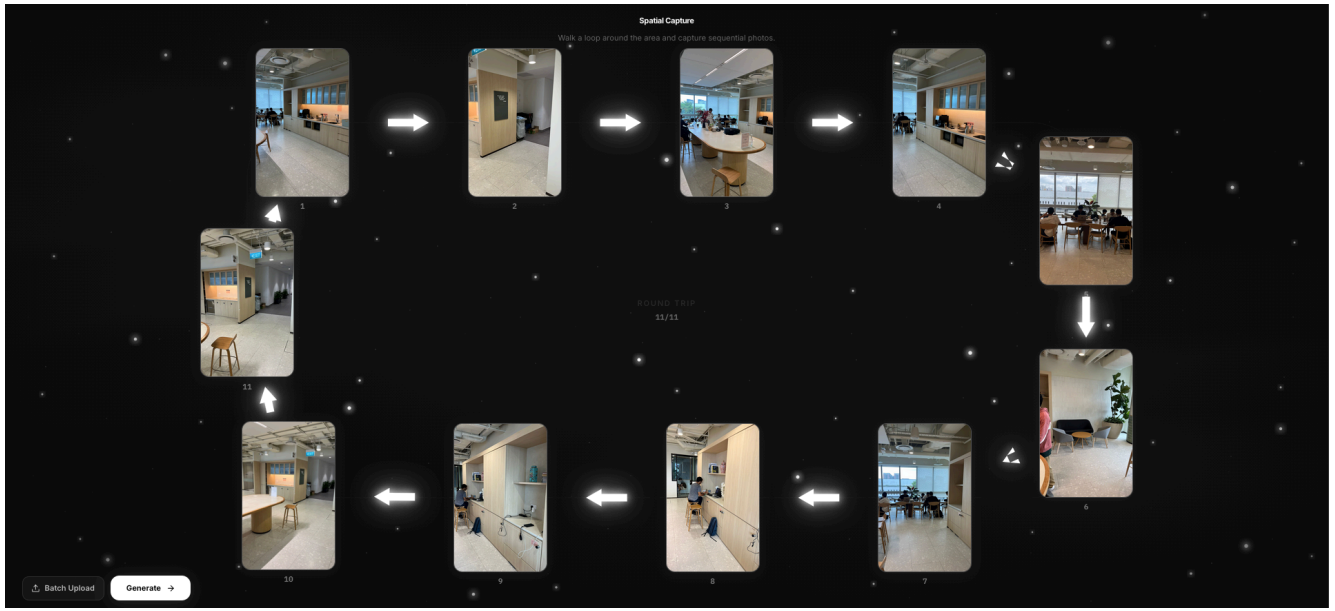


Source: Gemini Robotics-ER1.5 Demo

Our Approach: RPG

- Built on the Google AI Stack: Gemini 3, Nano Banana
- Input: sequential photographs from an indoor space
- Output:
    i. Structured floor map
    ii. Embodiment-aware task plans

Stage 1: Scene Decomposition



- Photos are sent as a single multimodal prompt to Gemini
- Model returns structured JSON for each image

Stage 1: Scene Decomposition

From these inputs..

Stage 1: Scene Decomposition

We get this:


```
{
  "node_name": "Office Pantry and Coworking Area",
  "static_anchors": [
    {
      "anchor_id": "pantry_counter_cabinet",
      "type": "wooden pantry cabinet",
      "description": "A large floor-to-ceiling wooden cabinetry unit with glass-front upper sections."
    },
    ...
  ],
  "dynamic_objects": [
    {
      "object_id": "coffee_espresso_machine",
      "type": "coffee machine",
      "description": "A black professional espresso machine on the main pantry countertop."
    }
  ],
  "navigable_edges": [
    {
      "edge_id": "hallway_exit_path",
      "description": "A corridor leading away from the pantry towards restrooms and building exits.",
      "visual_cue": "Illuminated green EXIT sign and overhead restroom pictograms."
    }
  ]
}
```

Stage 2: Layout Synthesis

- Step 1: Generate a text description of the room and all objects inside it
- Step 2: Generate a floor plan with Nano Banana

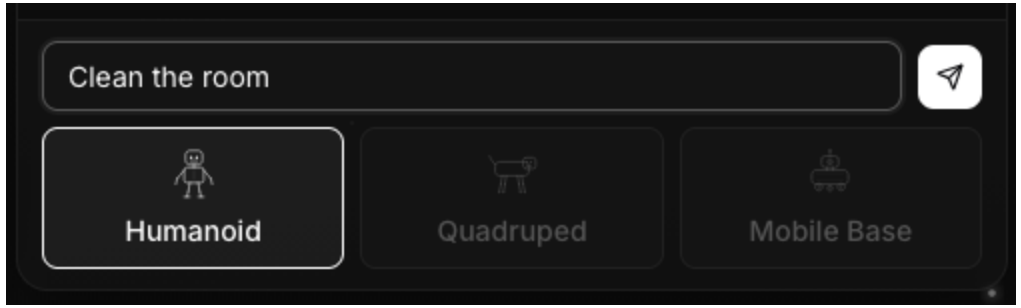
Stage 3: Object Localization

Create bounding boxes for identified objects for downstream planning tasks


```
[
  { "object_id": "pantry_counter_cabinet", "ymin": 5,  "xmin": 35, "ymax": 30, "xmax": 95 },
  { "object_id": "oval_communal_table",    "ymin": 35, "xmin": 30, "ymax": 65, "xmax": 70 },
  { "object_id": "coffee_espresso_machine","ymin": 8,  "xmin": 60, "ymax": 20, "xmax": 75 }
]
```

Embodiment-Aware Planning

- Goal: convert user intent into tasks that can be carried out by a robot
- Analogy: "Plan Mode" for Robots
    i. Generate sub-tasks to execute
    ii. List affected objects
    iii. List pre-requisite tasks
- Input: robot type, room entities and locations, user goal



Embodiment-Aware Planning

Account for different action spaces by maintaining a list of "skills" for each embodiment

```
ALLOWED_ACTIONS = {
    "humanoid": {
        "navigate", "move_to", "pick_up", "place", "grab", "carry", "open",
        "close", "push", "pull", "reach", "lift", "lower", "pour", "turn_on",
        # ... 70 actions
    },
    "quadruped": {
        "navigate", "move_to", "patrol", "inspect", "monitor", "push_low",
        "nudge", "follow", "guard", "detect", "sniff", "alert", "wait",
        # ... 25 actions
    },
    "mobile_base": {
        "navigate", "move_to", "sweep", "clean", "mop", "vacuum", "avoid",
        "patrol", "cover_area", "dock", "undock", "wait", "inspect_floor",
        # ... 21 actions
    },
}
```

Output

- TODO: Add a GIF of plan output here

Demo